

Reconstructing Human-Readable SQL from Optimized Logical Plans in DuckDB*

Lubin Ye

VU & Universiteit van Amsterdam
Amsterdam, The Netherlands
lubin.ye@student.uva.nl

Ilaria Battiston

CWI — Database Architectures
Amsterdam, The Netherlands
ilaria.battiston@cwi.nl

Peter Boncz

CWI — Database Architectures
Amsterdam, The Netherlands
boncz@cwi.nl

ABSTRACT

While SQL query engines transform declarative queries into optimized internal plan representations for efficient execution, the reverse direction—reconstructing readable SQL from such a plan—is rarely supported in open-source systems, yet is valuable for debugging optimizer behavior, explaining query transformations, and enabling portability across SQL systems. This work presents Logical Plan To SQL (LPTS), a DuckDB extension that converts a query’s optimized logical plan back into an equivalent, human-readable SQL string expressed as a sequence of named common table expressions (CTEs). LPTS implements a three-phase pipeline: a bottom-up traversal of the logical operator tree produces a dialect-agnostic abstract syntax tree (AST), which is then flattened into an ordered list of named CTEs and serialized to a `WITH . . . SELECT` SQL string. Supporting different output dialects—currently DuckDB, PostgreSQL, and Spark—enables portability across systems and serves cross-system use cases such as incremental view maintenance. The pipeline covers all the common logical operator types, including correlated subqueries, window functions, and (recursive) CTEs: evaluation on the TPC-H benchmark demonstrates empirical semantic round-trip correctness on all 22 queries (Q1–Q22). On the SQLStorm benchmark, covering 17,036 LLM-generated queries, LPTS achieves a 99.28% success rate on queries DuckDB executed deterministically (81.42% unconditionally).

1 INTRODUCTION

SQL query optimizers transform declarative queries into internal logical plans, applying a sequence of rewrites—filter pushdown, join reordering, subquery decorrelation—that progressively reshape the original query into a form optimized for execution. The result is an operator tree that precisely captures what the engine will execute, but that is disconnected from the SQL the user originally wrote. Inspecting optimizer behavior, explaining query transformations, or porting plan semantics to another SQL system all require reconstructing SQL from this internal representation—a step that no standard open-source tooling provides.

LPTS addresses this gap. A concrete motivating use case is OpenIVM [1], a SQL-to-SQL compiler for incremental view maintenance: OpenIVM uses DuckDB’s logical planner to derive SQL incremental queries for materialized views, and LPTS makes those compiler-generated plans human-readable and portable to other systems such as PostgreSQL. Another use case are SQL transpilers such as Coral and polyglot-sql, which translate queries across dialects but operate at the parse-tree level before optimization; they therefore cannot incorporate statistics-driven rewrites or other optimizer decisions into their output. By operating post-optimization

and being embedded directly in the engine, LPTS fills this gap. More generally, any time the optimizer rewrites a query into an equivalent but structurally different form, LPTS makes that transformation visible as readable SQL.

This work addresses three research questions:

RQ1. *What are the limitations of emitting SQL directly from logical plan nodes, how does an intermediate representation address them?*

RQ2. *How do query optimizers reshape logical plans for execution efficiency, and how does this affect SQL reconstruction?*

RQ3. *How can empirical semantic correctness and coverage of reconstructed SQL be evaluated across a diverse set of query patterns?*

The main contributions are:

- A complete three-phase pipeline (Logical Plan → AST → CTE List → SQL) showing empirical semantic round-trip correctness on all 22 TPC-H queries (Q1–Q22).
- Full compatibility with DuckDB’s optimizer pipeline: all optimizer passes are enabled, with each pass either handled transparently or supported by a dedicated AST node.

2 BACKGROUND AND RELATED WORK

DuckDB’s query execution model processes SQL through parsing, binding, logical planning, optimization, and physical execution [3]. The optimization phase produces a logical plan tree—a directed graph of operator nodes (scans, filters, joins, aggregates, and others) where each node carries fully resolved column references, predicate expressions, and table identifiers. This tree is the input to LPTS.

Reconstructing SQL from such a plan is non-trivial for several reasons.

Optimizer rewrites change the plan’s shape. Each optimizer rule modifies the plan to improve execution efficiency. For example, a filter predicate on a joined table may be moved below the join operator so that fewer rows are processed early—a transformation called filter pushdown. Similarly, an `ORDER BY` immediately followed by `LIMIT` may be combined into a single top-N operator. Reconstructing SQL from the optimized plan must reflect these structural changes; a pre-optimization plan would not faithfully represent what the engine executes.

Column names require reconstruction. Logical plan nodes reference columns by numeric identifiers rather than human-readable names. Translating these identifiers back to names requires maintaining a complete mapping throughout the traversal, which grows complex in the presence of joins and subqueries where multiple naming scopes interact.

Correlated subqueries are restructured. When a query contains a correlated subquery—a subquery that references a column from

*Source code available at <https://github.com/cwida/lpts>.

the outer query—the optimizer rewrites it into an equivalent join structure using the decorrelation technique described by Neumann and Kemper [8]. Reconstructing the original SQL, therefore, requires recognizing these join patterns and translating them back into the appropriate subquery or join syntax.

Related systems. The Umbra database system includes a web-based SQL explainer [7] that reconstructs SQL from its internal planner, demonstrating that the problem is tractable; however, its internals are not publicly documented. LinkedIn’s Coral [6] translates Hive QL plans across SQL dialects, and polyglot-sql [10] translates between dialects at the parse-tree level. Both operate before optimization and are unaffected by plan rewrites. LPTS occupies a different position in this design space: it operates on the post-optimization plan and produces per-operator CTEs that make each optimizer decision visible and individually executable. Codd’s relational model [2] underpins the bag-equivalence semantics that LPTS’s correctness evaluation verifies.

3 DESIGN AND IMPLEMENTATION

3.1 The LPTS Pipeline

LPTS converts an SQL query into an equivalent CTE-based SQL string through a three-phase pipeline (Figure 1), mapping each logical operator to a named CTE so that every step of the plan is individually readable and executable.

Phase 1 — Logical Plan to AST. The input query is parsed and planned by DuckDB, producing a logical operator tree. LPTS traverses this tree bottom-up, converting each operator into a corresponding node of a dialect-agnostic abstract syntax tree. Each AST node records the operator’s structural information—column references, filter predicates, join conditions—in a form independent of any target SQL dialect. A column name registry, built incrementally during the traversal, maps each internal numeric column reference to its human-readable name.

Phase 2 — AST to CTE List. The AST is visited in post-order (leaves before roots) and flattened into an ordered list of named CTE entries. Each AST node becomes one named CTE. Dependencies are expressed by name: a filter CTE reads from the preceding scan CTE, an aggregate CTE reads from the preceding filter, and so on. Post-order traversal guarantees that every CTE references only previously defined entries.

Phase 3 — CTE List to SQL. The ordered list is serialized into a WITH . . . SELECT SQL string. Each entry’s rendering step produces the operator-specific SQL fragment; dialect-specific decisions (e.g., table name qualification) are applied at this stage.

To illustrate, we can consider the query in Listing 1, which joins two tables on a key equality and applies one predicate on each side.

```
SELECT t.id, t2.score
FROM t JOIN t2 ON t.id = t2.id
WHERE t.val > 10 AND t2.score < 300;
```

Listing 1: Running example: a join query with one filter predicate per table.

```
-- Generate CTE SQL for a query:
```

```
PRAGMA lpts('SELECT t.id, t2.score FROM t JOIN t2 ON t
.id = t2.id
WHERE t.val > 10 AND t2.score < 300');
-- Output:
WITH scan_0(t1_id, t1_score) AS (
    SELECT id, score FROM test.main.t2 WHERE score <
    300
),
scan_1(t0_id) AS (
    SELECT id FROM test.main.t WHERE val > 10
),
join_2(t1_score, t0_id) AS (
    SELECT t1_score, t0_id FROM scan_0
    INNER JOIN scan_1 ON (t1_id = t0_id)
),
projection_3(t2_id, t2_score) AS (
    SELECT t0_id, t1_score FROM join_2
)
SELECT t2_id AS id, t2_score AS score FROM
    projection_3;
-- Round-trip correctness check (returns true):
PRAGMA lpts_check('SELECT t.id, t2.score FROM t JOIN
t2 ON t.id = t2.id
WHERE t.val > 10 AND t2.score <
300');
```

Listing 2: LPTS pragmas and example output for a join query with filter pushdown.

Listing 2 shows the LPTS user interface and example output on a join query with filter pushdown. The `lpts_check` primitive—the primary correctness oracle used in evaluation—executes both the original query and the LPTS-generated SQL and returns true if the result bags match.

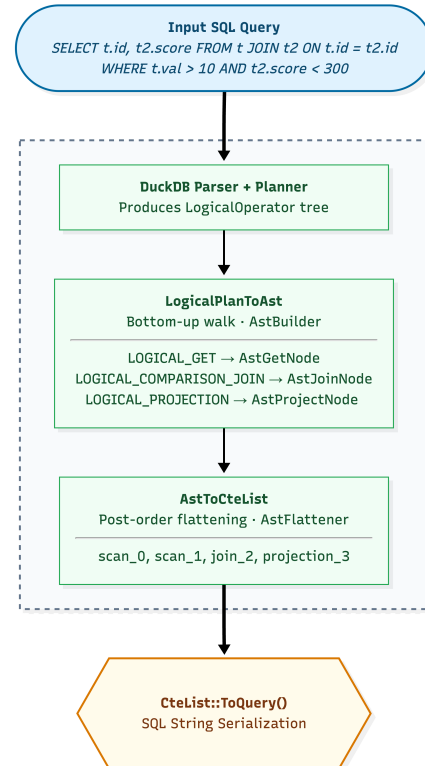


Figure 1: The LPTS conversion pipeline.

3.2 Intermediate AST Data Representation (RQ1)

An early prototype translated logical plan nodes directly into CTE entries in a single pass without an intermediate representation. While functional for basic operators, this design had three structural limitations.

Dialect coupling. Supporting multiple output dialects required scattering dialect-specific decisions throughout the tree-traversal code, coupling operator extraction to SQL rendering. The AST layer separates these concerns: the plan walker extracts operator semantics into typed node fields in a dialect-agnostic form, and dialect-specific rendering is confined to the serialization step. LPTS supports nine output dialects—DuckDB, PostgreSQL, Spark SQL, Hive, Trino/Presto, Snowflake, BigQuery, Redshift, and MySQL/MariaDB—and adding new ones requires no changes to the tree-traversal code. To illustrate the range of rendering decisions confined to this step: the DuckDB dialect (default) emits fully-qualified catalog.schema.table references; the PostgreSQL dialect drops the catalog and schema qualifiers to emit bare table names and remaps a small set of date/time function names to their PostgreSQL equivalents; and the Spark dialect applies backtick quoting to all identifiers, remaps a set of list and array function names to their Spark equivalents, and raises an error for window frame features not supported by Spark SQL.

Testability. The AST layer enables independent testing of the extraction phase. A dedicated PRAGMA `print_ast` command exposes the tree structure, allowing extraction regressions to be caught before they propagate to SQL output.

Extensibility. Adding a new operator follows a predictable three-step pattern: define a new AST node type with the operator’s fields; add a plan-walker case that populates those fields; add a CTE rendering method. Three operators related to materialized CTEs were added in a single iteration following this pattern, each requiring approximately 60–80 lines of new code.

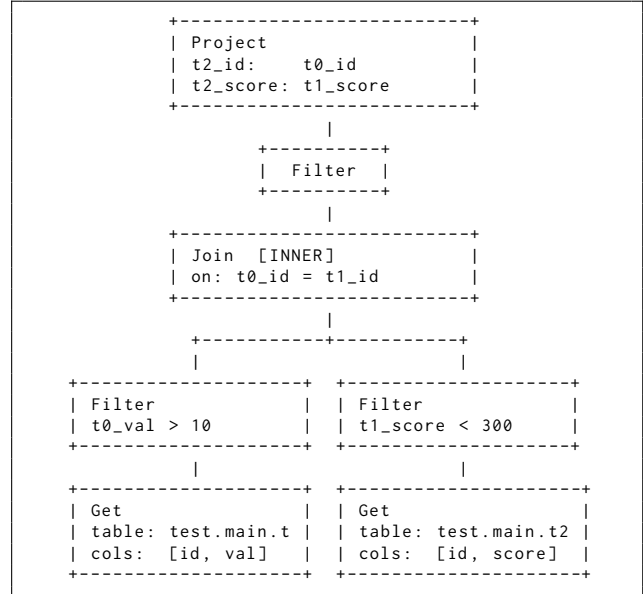
3.3 Optimizer Interactions (RQ2)

LPTS runs the full DuckDB optimizer pipeline with no passes disabled. Each optimizer rule either has no impact on how LPTS reconstructs SQL, or required adding support for the plan shape it introduces.

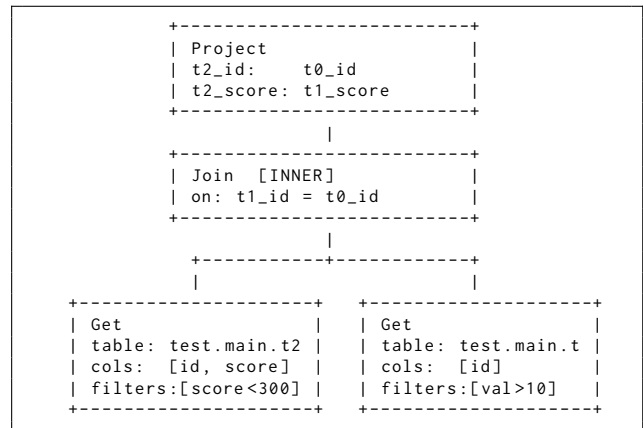
Some passes change the *structure* of the plan in ways that are directly visible in the output SQL. Filter pushdown relocates predicates onto individual scans, which LPTS serialises as WHERE clauses on the scan CTE. Top-N fusion combines an ORDER BY followed by a LIMIT into a single operator; LPTS renders both clauses together in one CTE. Column lifetime analysis prunes columns that are no longer needed above certain operators; LPTS accounts for this by emitting explicit output column lists rather than a wildcard, keeping CTE headers consistent with their bodies.

To illustrate, listings 3 and 4 show the logical plans produced for Listing 1 with FILTER_PUSHDOWN disabled and enabled, respectively. Without pushdown, both WHERE predicates remain as explicit Filter nodes above the Join, so the join receives the full, unfiltered input from both tables. With pushdown, each predicate is absorbed into the corresponding Get scan, and the join operates on pre-filtered rows. This reduces the number of rows reaching the join

operator. An additional benefit occurs at the storage layer: when the filter is part of the scan, DuckDB consults per-row-group Min-Max statistics before reading any data. If no value in a row-group can satisfy the predicate—for example, when the entire chunk has $\max(\text{val}) = 3$ and the filter is $\text{val} > 10$ —that chunk is skipped entirely, avoiding both reading and decompression. This data-skipping is where the performance gains happen.



Listing 3: Logical plan with FILTER_PUSHDOWN disabled. Both WHERE predicates remain as explicit Filter nodes above the Join, so the join receives the full unfiltered input from both tables.



Listing 4: Logical plan with FILTER_PUSHDOWN enabled. Each predicate is absorbed into its Get scan; the join receives pre-filtered rows, and MinMax statistics may allow entire row-groups to be skipped at the storage layer.

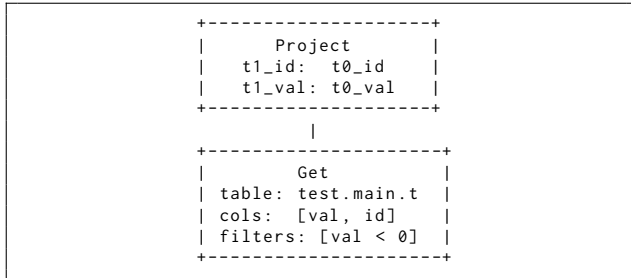
Other passes affect how WITH clause entries are represented in the plan. CTE inlining substitutes a CTE’s body at each reference point, producing ordinary nodes that need no special handling. The materialized CTE and common subplan passes instead share

a single computed result across multiple use sites; LPTS emits the shared result as a named WITH entry and references it by name at each use site, preserving the sharing in the output SQL.

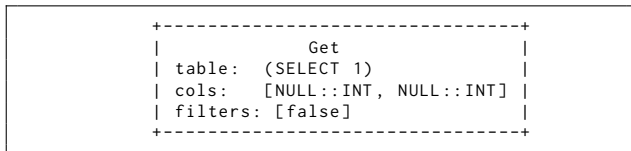
Statistics propagation can simplify expressions and prune branches of the plan that are statically determined to produce zero rows, replacing them with empty nodes; when this collapse is retained, LPTS handles it by emitting a synthetic scan of (SELECT 1) WHERE false, which yields zero rows with the correct column types.

Listings 5 and 6 show the plans for SELECT id, val FROM t WHERE val < 0 with STATISTICS_PROPAGATION disabled and enabled. With the pass disabled the predicate survives as an ordinary scan filter; with it enabled val < 0 is proven unsatisfiable and collapses the whole query to an empty scan. Unlike filter pushdown—which other optimizing engines could re-derive when the reconstructed SQL is executed—this pruning depends on stored statistics, so LPTS proves a predicate can never be satisfied, where the optimizer removes the corresponding subtree and replaces it with an empty result.

This data-dependence motivates a configurable guard. Because the empty-result is derived from the table’s current contents, the reconstructed SQL would silently become incorrect if the data later changed. LPTS therefore exposes a setting, lpts_enable_data_dependent_optimizers, defaulting to false. In the default conservative mode LPTS runs a local optimizer pass sequence that skips every pass whose decisions depend on current data, table statistics, cardinality estimates, row-group metadata, runtime dynamic filters, or common-subplan selectivity, so the generated SQL remains valid as the underlying data evolves; the data-dependent plan of Listing 6 is produced only when the setting is enabled. This local pass selection does not mutate DuckDB’s global disabled_optimizers setting—leaving concurrent connections unaffected—while any optimizer the user has explicitly disabled is still respected.



Listing 5: Logical plan without STATISTICS_PROPAGATION: the predicate remains as a scan filter.



Listing 6: Logical plan with STATISTICS_PROPAGATION enabled: the subtree is proven empty and collapsed to a synthetic empty scan (produced only when data-dependent optimizers are enabled).

3.4 Handling Non-Standard Query Shapes

Correlated subqueries. As described in Section 3.3, the optimizer rewrites correlated subqueries into explicit join structures. LPTS handles four such patterns. A correlated scalar subquery—for example, one that computes the average value for a matching group and compares it to an outer row—becomes a *left join* between the outer relation and the aggregated subquery result. The semantic equivalence of this transformation follows from the unnesting theorem of Neumann and Kemper [8]: a correlated expression (SELECT AVG(b) FROM T WHERE T.k = outer.k) can be replaced by a left join against (SELECT k, AVG(b) FROM T GROUP BY k), preserving result-set identity when the inner aggregation is unique on the join key. An EXISTS or NOT EXISTS predicate becomes a semi or anti join, respectively, keeping or discarding outer rows based on whether a matching inner row exists. An IN predicate involving a subquery becomes a mark join, which computes a boolean membership flag per outer row that is then used as a filter condition.

To illustrate: TPC-H Q17 asks for the average yearly extended price for parts ordered in small quantities. Its correlated subquery—“keep only lineitems where the quantity is less than 20% of the average quantity for that part”—is decorrelated into a join between the lineitem table and a per-part average, which LPTS reconstructs as a LEFT JOIN with an arithmetic condition. TPC-H Q21 uses both EXISTS and NOT EXISTS in the same query, exercising semi and anti join reconstruction simultaneously.

Materialized CTEs. A SQL query that defines a WITH clause entry referenced more than once causes the planner to emit a materialized CTE plan node with a corresponding reference node at each use site. LPTS handles this by generating a top-level WITH clause entry for the CTE body and emitting name references at each use site. TPC-H Q15 exercises this path: it defines a CTE that computes per-supplier revenue, used once to find the maximum revenue value and again to join with the supplier table.

4 EVALUATION

4.1 Experimental Setup

Experiments use DuckDB v1.5. TPC-H data is generated using DuckDB’s built-in data generator at scale factor 0.01 for TPC-H and scale factor 1 for SQLStorm, using DuckDB’s built-in TPC-H extension [4]. Correctness is evaluated using the round-trip check where both the original query and the LPTS-generated SQL are executed and their result bags compared, with true returned on a match.

4.2 TPC-H Benchmark (RQ3)

LPTS correctly round-trips all 22 standard TPC-H queries (Q1–Q22) [5]. The suite exercises the full range of operators covered by LPTS: multi-way joins up to 8 tables (Q5, Q8); all four correlated subquery patterns—scalar (Q2, Q17), EXISTS/NOT EXISTS (Q4, Q21), and mark-join IN (Q16, Q22, Q20); materialized CTEs (Q15); outer joins with NOT LIKE conditions (Q13); and CASE WHEN expressions within aggregates (Q8, Q12, Q14). The most complex queries are Q2 (5-way join combined with a correlated scalar subquery) and Q21 (EXISTS and NOT EXISTS in the same query); both pass. Six

Table 1: SQLStorm results (17,036 queries, sf=1). LPTS achieves 99.28% success rate on queries executed deterministically (excluding timeouts and DuckDB-side errors) confirming broad coverage across diverse query patterns.

Category	Count	%
Success	13,870	81.42
Nondeterministic	1,976	11.60
DuckDB error	1,007	5.91
LPTS error	100	0.59
Timeout	83	0.49

queries (Q2, Q4, Q17, Q20, Q21, Q22) were initially unsupported and were added after decorrelation support was implemented.

4.3 SQLStorm Benchmark (RQ3)

The SQLStorm benchmark [9] consists of 17,036 LLM-generated TPC-H schema queries covering a wide range of operator combinations, aggregate functions, window functions, and subquery patterns. It is well-suited to LPTS evaluation because its breadth exposes edge cases in expression serialization and operator interactions that hand-written benchmarks do not reach. Queries run at scale factor 1 with a 30-second timeout on 16 threads.

Table 1 shows the results. The central finding is that LPTS produces empirically semantic equivalent SQL for every query it processes to completion, as verified by round-trip bag comparison on the specific dataset test instance: with a single exception across 17,036 queries, all completed runs yield a correct result.

The 1007 *DuckDB errors* are queries that DuckDB itself fails to execute, independent of LPTS; they reflect limitations of the LLM-generated query workload rather than LPTS behavior. The 100 *LPTS errors* stem from a small number of serialization edge cases: certain optimizer-introduced expression forms that are not yet handled in all contexts, and column naming conflicts that arise in complex multi-aggregate queries.

On the performance side, we compare the wall-clock time of the original query against that of the LPTS-reconstructed query (excluding one-off generation costs such as plan-walking, AST construction, and CTE serialization). The two are effectively identical, averaging 137.9 ms and 136.1 ms (medians 47.2 ms and 48.1 ms) respectively, over 13,870 successful queries. This is expected: because `lpts_exec` hands the generated SQL back to DuckDB as an ordinary query, it is re-parsed, re-bound, and *re-optimized* before execution, so both figures measure DuckDB executing two empirically equivalent queries.

5 DISCUSSION AND CONCLUSION

5.1 Answers to Research Questions

RQ1 — Pipeline Design. Emitting SQL directly from logical plan nodes couples plan traversal to rendering, making multiple dialects unwieldy and preventing independent testing of each extraction. The AST layer separates these concerns: operator semantics are extracted into typed, dialect-agnostic node fields; rendering and

dialect decisions are confined to the serialization step. This separation enabled dialect support without modifying the tree-traversal code and enabled AST-level tests before they affect SQL output.

RQ2 — Optimizer interactions. DuckDB’s optimizers reshape the logical plan in ways that change SQL reconstruction, e.g., `FILTER_PUSHDOWN` moves predicates from post-join filter nodes into individual table scans. LPTS operates on the post-optimisation logical plan, so its SQL output reflects the optimiser’s structural decisions. By default it skips data-dependent passes (such as statistics-driven pruning), keeping the reconstructed SQL valid across data changes, with an opt-in setting to retain them when desired.

RQ3 — Correctness and coverage. Round-trip evaluation of the original and LPTS-generated query by comparing result bags provides a rigorous empirical semantic oracle that does not depend on syntactic equivalence. TPC-H exercises 22 structurally diverse queries, all of which pass. SQLStorm extends coverage to 17,036 LLM-generated queries: of the 13,970 queries DuckDB executed deterministically (excluding timeouts and DuckDB-side errors), 99.28% succeed.

5.2 Conclusion

LPTS demonstrates that reconstructing readable, empirically semantical equivalent SQL from DuckDB’s post-optimization logical plan is feasible and broadly correct. The three-phase pipeline (Logical Plan $\rightarrow$ AST $\rightarrow$ CTE List $\rightarrow$ SQL) round-trips all 22 TPC-H queries. The intermediate AST layer enables clean dialect separation, independent testability and a predictable per-operator extension pattern. The pipeline runs with every DuckDB optimizer pass enabled. On 17,036 SQLStorm queries, LPTS succeeds on 99.28% of those DuckDB executed deterministically, with the remaining failures confined to DuckDB-side execution errors, a few serialization edge cases, and non-trivial recursive CTE patterns not yet implemented.

5.3 Limitations and Future Work

Several directions remain open.

Serialization edge cases in SQLStorm failures. The 100 LPTS errors in Table 1 fall into two primary categories. (1) The larger group involves column-naming conflicts in complex multi-aggregate queries: when several aggregates in the same CTE produce identically named output columns, the positional naming scheme generates duplicate CTE column headers, which DuckDB rejects. (2) The smaller group consists of optimizer-introduced expression forms—such as certain `CASE WHEN` rewrites and lambda-lifted expressions—that appear only in edge-case query shapes and are not yet fully serialized in all operator contexts. Both are tractable: name conflicts require a collision-detection pass over generated CTE headers, while missing expression forms each require a targeted serialization case.

Cross-dialect plan optimization. LPTS reconstructs SQL from DuckDB’s post-optimization plan, but it does not verify that the resulting string is well-optimized for the target dialect’s own engine. A concrete experiment would toggle individual DuckDB optimizer passes—filter pushdown, join reordering, decorrelation—before generating the output SQL, then execute the result on PostgreSQL and

Spark SQL and compare their native execution plans and wall-clock runtimes against executing the original query directly. This would isolate which rewrites transfer across engines and which are redundant given the target engine’s own optimizer. The current position that a plan optimized by DuckDB yields good SQL for other dialects is still untested, since dialect queries cannot be validated with the current `lpts_check` as it would need to be executed on the specific dialect databases. A useful direction would be to execute the LPTS output on each target engine and compare its native plan and runtime against the same query, establishing empirically whether the reconstructed SQL is actually optimized for that dialect rather than merely correct.

Human-readable column naming. LPTS currently names CTEs and their columns positionally (`scan_0`, `scan_1`, `projection_3`, and so on), which is unambiguous but hard to read, and is also the source of the column-naming conflicts behind several of the LPTS errors reported in Table 1. A promising direction is to couple a small, CPU-only language model—one that can run embedded within the DuckDB process without network access—to perform a renaming pass over the generated CTE headers and column aliases, assigning descriptive, collision-free names derived from the underlying table and expression semantics. This would improve readability while resolving the naming conflicts identified above, and would preserve the self-contained, embeddable trait of the extension.

Plan-level equivalence checking. The current correctness oracle compares result bags, which establishes equivalence only on the tested database instance. A complementary mechanism would feed the LPTS-generated SQL back into DuckDB’s planner and compare the resulting logical plan against the original plan LPTS started from, checking that the reconstruction round-trips at the plan level rather than only at the result level. Plan-level agreement would provide instance-independent evidence that LPTS preserves the optimizer’s structure, rather than merely reproducing one query result on one database.

Full DuckDB unit test suite. The current evaluation uses TPC-H (22 queries) and SQLStorm (17,036 queries). DuckDB’s own SQL logic test suite covers a larger and more systematic space of edge cases—type coercions, null semantics, set operations, lateral joins, and others—making it a natural next test surface. Running `lpts_check` over the full suite would provide coverage of operators and expression forms that neither TPC-H nor the LLM-generated SQLStorm queries reliably exercise.

ACKNOWLEDGMENTS

This work was carried out as a Master’s research project in Computer Science at VU Amsterdam / Universiteit van Amsterdam, supervised by Ilaria Battiston and Peter Boncz at CWI — Database Architectures, Amsterdam.

REFERENCES

- [1] Ilaria Battiston, Kriti Kathuria, and Peter Boncz. 2024. OpenIVM: a SQL-to-SQL Compiler for Incremental Computations. In *Companion of the 2024 International Conference on Management of Data (SIGMOD-Companion ’24)*. ACM, Santiago, Chile, 1–4. <https://doi.org/10.1145/3626246.3654743>
- [2] Edgar F. Codd. 1970. A Relational Model of Data for Large Shared Data Banks. *Commun. ACM* 13, 6 (1970), 377–387. <https://doi.org/10.1145/362384.362685>
- [3] DuckDB Labs. 2024. DuckDB: An In-Process Analytical Database. <https://github.com/duckdb/duckdb>.
- [4] DuckDB Labs. 2025. DuckDB TPC-H Extension. https://duckdb.org/docs/current/core_extensions/tpch.
- [5] DuckDB Labs. 2025. TPC-H Query Definitions (DuckDB implementation). <https://github.com/duckdb/duckdb/tree/main/extension/tpch/dbgen/queries>.
- [6] LinkedIn Engineering. 2020. Coral: A SQL Translation, Analysis, and Rewrite Engine. <https://github.com/linkedin/coral>.
- [7] Thomas Neumann et al. 2024. Umbra SQL Interface. <https://umbra-db.com/interface/>.
- [8] Thomas Neumann and Alfons Kemper. 2015. Unnesting Arbitrary Queries. In *Proceedings of the 26th GI-Workshop on Foundations of Databases (BTW 2015)*. Gesellschaft für Informatik, Hamburg, Germany, 383–402. https://www.btw-2015.de/res/proceedings/Hauptband/Wiss/Neumann-Unnesting_Arbitrary_Query.pdf
- [9] Tobias Schmidt, Viktor Leis, Peter Boncz, and Thomas Neumann. 2025. SQLStorm: Taking Database Benchmarking into the LLM Era. *Proc. VLDB Endow.* 18, 11 (2025), 4144–4157. <https://doi.org/10.14778/3749646.3749683>
- [10] tobilg. 2022. polyglot-sql: Translating SQL Between Dialects. <https://github.com/tobilg/polyglot>.